

# **cqs — Code Quality Studio CLI**

18 stacks · 530 rules · offline · CI-ready · Node 18+

INSTALL FROM NPM (ANYWHERE)  
**npm install -g cqs-audit**  
[npmjs.com/package/cqs-audit](https://npmjs.com/package/cqs-audit)

✓ Verify: **cqs --list-stacks**

v2.1.0

## 1 · Install & Everyday Use

Install → verify → run your first audit

### INSTALL FROM NPM (PUBLIC — ANY MACHINE)

```
# Install globally from npm registry
npm install -g cqs-audit
```

```
# Verify
cqs --help
```

 Package: **cqs-audit** on [npmjs.com/package/cqs-audit](https://npmjs.com/package/cqs-audit)

### CORE COMMANDS

| COMMAND                                            | DESCRIPTION                                 |
|----------------------------------------------------|---------------------------------------------|
| <b>cqs .</b>                                       | Analyse current folder — auto-detects stack |
| <b>cqs ./tests/</b>                                | Analyse a specific folder                   |
| <b>cqs ./tests/ --stack playwright</b>             | Force a specific stack                      |
| <b>cqs ./tests/ --severity critical</b>            | Show only critical findings                 |
| <b>cqs ./tests/ --output json &gt; report.json</b> | Save JSON report for CI or sharing          |
| <b>cqs ./tests/ --output summary</b>               | One-line status — great for CI logs         |
| <b>cqs ./tests/ --open</b>                         | Run analysis + open HTML report in browser  |
| <b>cqs --read-report report.json</b>               | Print summary of a saved JSON report        |
| <b>cqs --read-report report.json --open</b>        | Open saved report as HTML in browser        |
| <b>cqs --list-stacks</b>                           | Show all 18 available stacks + IDs          |
| <b>cqs --help</b>                                  | Show help text and version number           |

### ALL 18 STACKS (--STACK <ID>)

#### UI AUTOMATION

-  playwright
-  playwright\_java
-  playwright\_python
-  selenium\_java
-  selenium\_csharp
-  cypress

#### MOBILE / MODEL-BASED

-  appium\_java
-  toscaxml

#### API TESTING

-  restassured
-  karate
-  pytest\_api
-  postman

#### APPLICATION CODE

-  java\_api
-  java\_frontend
-  typescript
-  ts\_frontend
-  python\_api
-  python\_frontend

2 · AI-Assisted Review

Set an API key once, then every run can include an AI second eye

🤖 AI SETUP — ANTHROPIC & OPENAI

ANTHROPIC (CLAUDE)

```
# 1. Key from console.anthropic.com
# 2. Persist it in your shell profile
echo 'export ANTHROPIC_API_KEY=sk-ant-xxx' >> ~/.zshrc
source ~/.zshrc
# 3. Run — key picked up automatically
cqs ./tests/ --ai anthropic
```

Set the key **once** — every later run reads it from the environment.

OPENAI (GPT)

```
# 1. Key from platform.openai.com
echo 'export OPENAI_API_KEY=sk-xxx' >> ~/.zshrc
source ~/.zshrc
# 2. Run (default model gpt-4o-mini)
cqs ./tests/ --ai openai
```

☁️ AI SETUP — BEDROCK & VERTEX

AWS BEDROCK

```
echo 'export AWS_REGION=us-east-1' >> ~/.zshrc
echo 'export AWS_ACCESS_KEY_ID=AKIAxxx' >> ~/.zshrc
echo 'export AWS_SECRET_ACCESS_KEY=xxx' >> ~/.zshrc
source ~/.zshrc
cqs ./tests/ --ai bedrock
```

GOOGLE VERTEX AI

```
# Service-account JSON from the GCP console
echo 'export VERTEX_PROJECT=my-proj' >> ~/.zshrc
echo 'export VERTEX_LOCATION=us-central1' >> ~/.zshrc
echo 'export GOOGLE_APPLICATION_CREDENTIALS=~/.sa.json' >> ~/.zshrc
source ~/.zshrc
cqs ./tests/ --ai vertex
```

INLINE (CI — no env var)

```
cqs ./tests/ --ai anthropic --api-key sk-ant-xxx
cqs ./tests/ --ai openai --api-key sk-xxx
```

🔥 CORE AI COMMANDS (AFTER KEY IS SET)

# Audit with AI second-eye review

```
cqs ./tests/ --stack playwright --ai anthropic
cqs ./tests/ --stack playwright --ai openai
cqs ./tests/ --stack playwright --ai bedrock
cqs ./tests/ --stack playwright --ai vertex
```

# Review only the critical files

```
cqs ./tests/ --ai anthropic --severity critical
```

# AI review + open HTML report

```
cqs ./tests/ --ai anthropic --open
```

# AI review + save JSON

```
cqs ./tests/ --ai anthropic --output json > r.json
```

PICK A STRONGER MODEL

```
cqs ./tests/ --ai anthropic --model claude-sonnet-5
cqs ./tests/ --ai openai --model gpt-4o
```

AI review runs **after** the offline rules and only on files that already have critical or warning findings.

### 3 · CI/CD, Output Formats & Packaging

Gate your pipeline, choose a report format, publish new versions

#### CI / CD INTEGRATION

##### GITHUB ACTIONS

```
- name: Audit test quality
  run: cqs ./tests/ --output summary
  # exit 1 on criticals = failed step
```

##### GITLAB CI

```
quality-gate:
  script: cqs ./tests/ --output json >
  r.json
  artifacts: { paths: [r.json] }
```

##### PRE-COMMIT HOOK

```
cqs . --severity critical --output summary
|| exit 1
```

#### OUTPUT FORMATS & ADVANCED

**pretty** Colour terminal — scores, findings, bars (default)

**json** Machine-readable — pipe to file for CI artefacts

**summary** One-liner: score 90 · 5 critical · 12 warning

##### ADVANCED COMMANDS

```
# Filter by category
cqs ./tests/ --category reliability

# Multiple paths
cqs ./src/tests/ ./e2e/ --stack playwright

# Plain text (no ANSI — for log files)
cqs ./tests/ --no-color

# Save + open later
cqs ./tests/ --output json > r.json
cqs --read-report r.json --open
```

4 · IDE & AI Agent Integration

Run cqs from Cursor, or expose it as a tool your AI agent can call

USING CQS WITH CURSOR IDE

OPTION 1 — Terminal inside Cursor

```
# Open terminal in Cursor (Ctrl+`), then
run normally
cqs ./tests/ --stack playwright --open
# HTML report opens in your default browser
```

OPTION 2 — Cursor Task (tasks.json)

```
// .vscode/tasks.json
"tasks": [{
  "label": "cqs audit",
  "type": "shell",
  "command": "cqs ${workspaceFolder}/tests/ --open",
  "group": "test"
}]
```

OPTION 3 — Cursor AI + cqs together

1. Run: `cqs ./tests/ --output json > r.json`
2. Open `r.json` in Cursor
3. Ask Cursor AI: "Fix the critical findings in this report"
4. Cursor reads the JSON + your test files and applies fixes

MCP SERVER — CQS AS AN AI TOOL

BUILD THE MCP SERVER

```
# From the cqs CLI source directory
cd playwright-quality-studio/cli
node build-mcp.mjs
# → produces dist/cqs-mcp.js
```

CONFIG — SAME JSON FOR BOTH CLIENTS

```
{
  "mcpServers": {
    "cqs": {
      "command": "node",
      "args": ["/abs/path/dist/cqs-mcp.js"]
    }
  }
}
```

Cursor → `.cursor/mcp.json`

Claude Code → `~/.claude/settings.json`

MCP TOOLS EXPOSED

- `cqs_audit` — audit a path, returns findings
- `cqs_list_stacks` — list all 18 supported stacks
- `cqs_read_report` — summarise a JSON report file

Once registered, ask the AI: "Audit my tests folder" — the AI calls `cqs_audit` automatically.

CLAUDE CODE /CQS SKILL

INSTALL THE SKILL (one-time)

```
# Create the skill directory
mkdir -p ~/.claude/skills/cqs

# Copy SKILL.md from the cqs repo
cp cli/bin/cqs-skill.md
~/.claude/skills/cqs/SKILL.md

# Or download directly (if cqs-audit is
installed globally)
cqs --list-stacks # verify cqs is working
```

USE IN CLAUDE CODE SESSION

```
# In any Claude Code session, just type:
/cqs

# Claude will ask you for the path, then
run the audit
# and summarise findings directly in the
conversation
```

EXAMPLE PROMPTS WITH SKILL

"Audit my tests and fix all critical issues"

"Run cqs on ./e2e/ and summarise what's wrong"

"Check my Playwright tests for quality issues"

"Review ./tests/ with AI — use Anthropic"

SKILL FILE LOCATION

```
~/.claude/skills/cqs/SKILL.md
```

Claude Code auto-loads skills from `~/.claude/skills/` — no restart needed after adding SKILL.md.